

Prepared by:

Muhammad Sarim

Roll No: 00186281

Class Timing: Friday 6–9 PM

AIDD 30 Days Challenge – Task 2

Part A – Theory (Short Questions)

Nine Pillars Understanding

Question 1:

Why is using AI Development Agents, like Gemini CLI, for repetitive setup tasks better for your growth as a system architect?

Answer: Employing the use of AI Agents for repetitive tasks, like Gemini CLI, keeps developers away from the mental fatigue that might arise from doing boring tasks. Without AI, the developer often works in solitude, concentrating his full time on one product. AI does the repetition in setup, coding, folder structure, and environment configuration so that the developer can pay more attention to high-level architecture, scalability, and feature planning. This collaboration with AI enables developers to construct stronger and efficient system architecture.

Question 2:

Describe how the Nine Pillars of AIDD help a developer grow into an M-Shaped Developer?

Answer: The Nine Pillars act as guidelines for this AI-driven development. Some key components include:

- 1) SDD (Spec-Driven Development): A full specification created by developers guides AI on how to perform tasks.
- 2) MCP Standard: Ensures that AI interprets the specifications correctly.
- 3) AI CLI Coding Agent: AI converts specifications into functional code.
- 4) AI-First IDEs: This allows for automatic file creation and folder structuring by AI itself.

It enables developers to work across many domains such as frontend, backend, design, and more—thereby being M-Shaped Developers with depth in multiple skills.

Vibe Coding vs. Specification-Driven Development

Question 1:

Why does Vibe Coding usually create problems after a week?

Vibe Coding: Vibe coding occurs when developers code with AI without structure or specifications. The application goes smoothly during the first times of development, but after some time, the developer becomes unaware of what the AI wrote, or where functions are defined. The bugs become not debuggable, and the architecture is quite unstable.

Question 2:

Vibe Coding vs. Specification-Driven Development?

Q: In Vibe Coding, AI is called upon to code without clear guidance. Specification-Driven Development provides a full design on the final product, much like following a syllabus. The AI simply carries out the tasks as specified to ensure reliable results with minimal errors.

Architecture Thinking

Question 1:

How does architecture-first thinking change the role of a developer in AIDD?

Answer: The developers evolve from manual coders to system architects. They design blueprints, then delegate tasks to AI and let it resolve bugs on its own. The developer focuses on the high-level design of the system rather than writing each and every line of code.

Question 2:

Explain why developers must think in layers and systems, rather than raw code?

Answer: Layered thinking ensures scalable and maintainable systems. AIDD operates in layers: specifications, logic, AI coding, testing, and implementation. Thinking in layers clarifies responsibilities, reduces risks, and speeds up development.

Part B - Practical Task

Task: Using any AI CLI tool, generate a 1-paragraph specification for an email validation function. My Prompt Screenshot:

```
> Command: Generate a one paragraph specifications for an "email validation function." Requirements: The function must check that "@" is present. It must check that email end with a valid domain extension like (.net, .com, .pk, .org etc.). It must return a clear, human understandable error messages for each type of validation failures. Keep the specification technical but readable for everyone. Output format: Produce a well structured paragraph describing the functional specifications.
```

My Prompt Text:

Command: Write one-paragraph specifications for an "email validation function."
Requirements: The function should check that "@" is present. It should check that email ends with a valid domain extension like (.net, .com, .pk, .org etc.). It should return clear human-understandable error messages for each type of validation failure. Make the specification technical yet understandable by all. Output format: Well-structured paragraph describing functional specifications.

Gemini CLI Response Screenshot:

```
▣ The email validation function shall accept a string input representing an email address and perform a series of checks to ensure its syntactic correctness. Specifically, it must verify the presence of a single "@" symbol separating the local part from the domain. Furthermore, the function will validate the domain against a predefined list of top-level domains (TLDs) such as .com, .net, .org, .pk, ensuring the email address terminates with a recognized and valid extension. Upon encountering any validation failure, the function shall return a distinct, human-readable error message precisely indicating the nature of the invalidity, such as "Missing '@' symbol," "Invalid domain format," or "Unsupported top-level domain," to facilitate clear user feedback and debugging.
```

The email validation function takes a string input representing an email address and does a series of checks to determine whether the email address is syntactically correct. Specifically, it needs to check for the presence of one and only one "@" symbol separating the local part from the domain. The domain will also be checked against a list of predefined TLDs such as .com, .net, .org, .pk, which means the email address is terminated by a known and valid extension. For every failure in the validation, the function returns a unique, human-readable error message exactly describing the nature of its invalidity, such as "Missing '@' symbol," "Invalid domain format," or "Unsupported top-level domain," for clear user feedback and debugging.

Part C – MCQs

1. What is the main purpose of Spec-Driven Development?

- A. Speed up coding
- B. Clearly define the requirements before coding starts.
- C. Remove developers
- D. Avoid documentation

My Answer: B

2. What is the biggest mindset shift in AI-Driven Development?

- A. Writing more code manually
- B. System thinking and clarity of instruction
- C. Memorizing more syntax
- D. Working without tools My

Answer: B

3. Biggest failure of Vibe Coding?

- A. AI stops responding
- B. Architecture becomes hard to extend
- C. Code runs slow
- D. Fewer comments written

My Answer: B

4. Primary benefit to using AI CLI agents, like Gemini CLI?

- A. They replace the developer completely.
- B. Handle repetitive tasks so dev focuses on design & problem-solving
- C. Make coding faster but less reliable
- D. Make coding optional

My Answer: B

5. What is an M-Shaped Developer?

- A. Knows little about everything
- B. Deep in only one field
- C. Deep skills in multiple related domains
- D. Works without AI tools

Answer: C By the end of this

Assignment: After this assignment,

I'm very much aware of some new terms and some new techniques. Learned what NINE PILLARS of AIDD are, what the qualities of the M-Shaped Developer are, and how vibe coding is not effective without SDD (Spec-Driven Development).